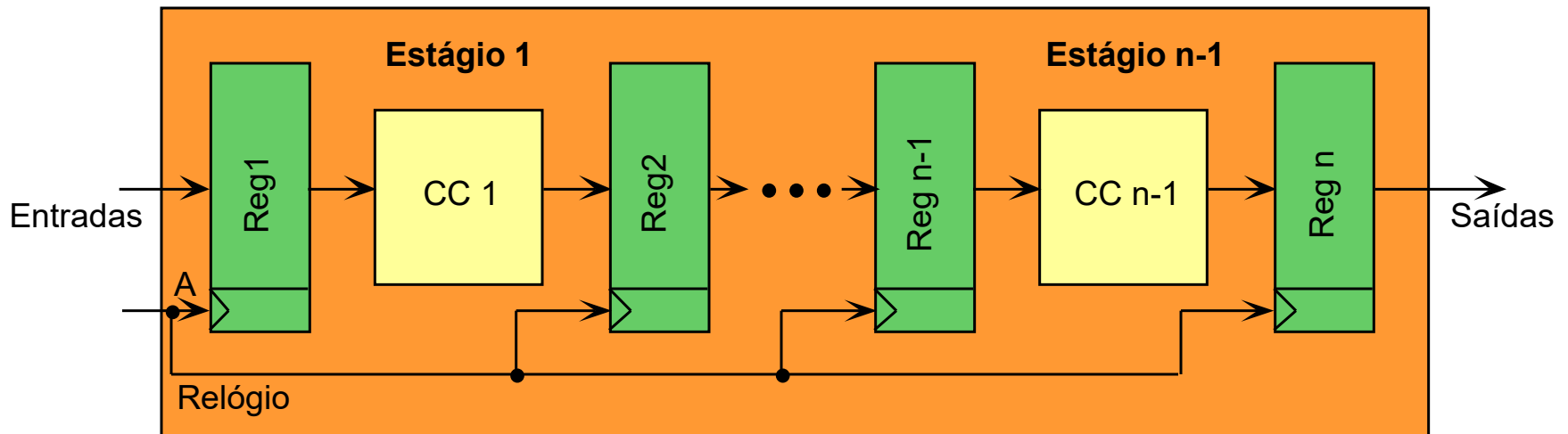

Implementação de um Subconjunto Multiciclo do Processador MIPS – MIPS_S

Fernando Moraes (09/10/2006)

Última alteração - Ney Calazans, 23/07/2026

Descrição RTL de HW Multiciclo

- Cada estágio realiza uma parte do trabalho
- Registradores → usados para isolar os resultados as chamadas “barreiras temporais”



Especificação da Organização MIPS_S

- **Dá suporte a 37 das 168 instruções da ISA MIPS-32™**
 - Aritméticas (5): ADDU, SUBU, MULTU, DIVU, ADDIU
 - Lógicas (7): AND, OR, XOR, NOR, ANDI, ORI, XORI
 - Deslocamento de bits (6): SLL, SLLV, SRA, SRAV, SRL, SRLV
 - Acesso à Memória (4): LBU, LW, SB, SW
 - Teste (4): SLT, SLTU, SLTI, SLTIU
 - Controle de Fluxo (8): BEQ, BGEZ, BLEZ, BNE, J, JAL, JALR, JR
 - Miscelâneas (3): MFHI, MFLO, LUI
- **Maior parte das instruções executa em 4 ciclos de relógio**
- **LW e LBU executam em 5 ciclos de relógio**
- **MULTU e DIVU executam em 67 ciclos de relógio**

Especificação da Organização MIPS_S

- Ao lado, a Table 2-1 do manual **MIPS32™ Architecture For Programmers Volume I: Introduction to the MIPS32™ Architecture**, publicado em 2001 pela empresa MIPS Technologies
- Anotações em cores são do Autor (ver Legenda)
- No mesmo manual, ver a Table 2-2, com as 90 instruções que a ISA MIPS64, compatível com a MIPS32, acrescenta à ISA MIPS32

Chapter 2 The MIPS Architecture: An Introduction

2.5.1 List of MIPS32 Instructions

Table 2-1 lists of those instructions included in the MIPS32 ISA.

Legenda (Total ->168 instruções):

Amarelo - MIPS_S (37)

Azul - MIPS_S_Cp0 (17)

Vermelho - Cp1, FPU (52)

Verde - Cp2 (12)

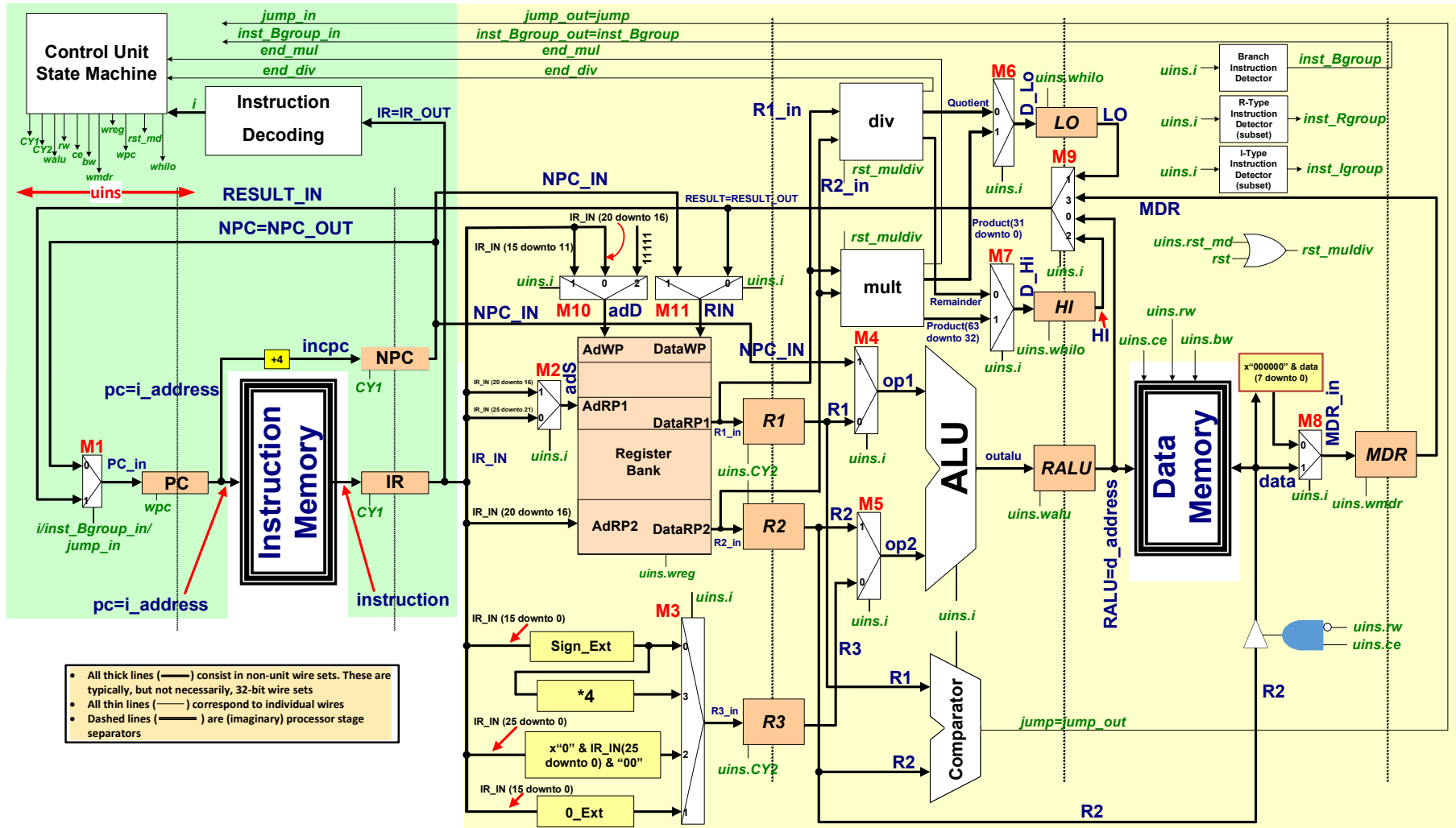
Sobram - 50

Table 2-1 MIPS32 Instructions

ABS.D	ABS.S	ADD	ADD.D	ADD.S	ADDI	ADDIU	ADDU
AND	ANDI	BC1F	BC1FL	BC1T	BC1TL	BC2F	BC2FL
BC2T	BC2TL	BEQ	BEQL	BGEZ	BGEZAL	BGEZALL	BGEZL
BGTZ	BGTZL	BLEZ	BLEZL	BLTZ	BLTZAL	BLTZALL	BLTZL
BNE	BNEL	BREAK	C.cond.D	C.cond.S	CACHE	CEIL.W.D	CEIL.W.S
CFC1	CFC2	CLO	CLZ	COP2	CTC1	CTC2	CVT.D.S
CVT.D.W	CVT.S.D	CVT.S.W	CVT.W.D	CVT.W.S	DIV	DIV.D	DIV.S
DIVU	ERE1	FLOOR.W.D	FLOOR.W.S	J	JAL	JALR	JR
LB	LBU	LDC1	LDC2	LH	LHU	LL	LUI
LW	LWC1	LWC2	LWL	LWR	MADD	MADDU	MFC0
MFC1	MFC2	MFHI	MFLO	MOV.D	MOV.S	MOVF	MOV.F.D
MOVES	MOVN	MOVN.D	MOVN.S	MOVT	MOVT.D	MOVT.S	MOVZ
MOVZ.D	MOVZ.S	MSUB	MSUBU	MTC0	MTC1	MTC2	MTHI
MTLO	MUL	MUL.D	MUL.S	MULT	MULTU	NEG.D	NEG.S
NOR	OR	ORI	PREF	ROUND.W.D	ROUND.W.S	SB	SC
SDC1	SDC2	SH	SLL	SLLV	SLT	SLTI	SLTIU
SLTU	SQRT.D	SQRT.S	SRA	SRAV	SRL	SRLV	SSNOP
SUB	SUB.D	SUB.S	SUBU	SW	SWC1	SWC2	SWL
SWR	SYNC	SYSCALL	TEQ	TEQI	TGE	TGEI	TGEIU
TGEU	TLBP	TLBR	TLBWI	TLBWR	TLI	TLTI	TLTIU
TLTU	TNE	TNEI	TRUNC.W.D	TRUNC.W.S	WAIT	XOR	XORI

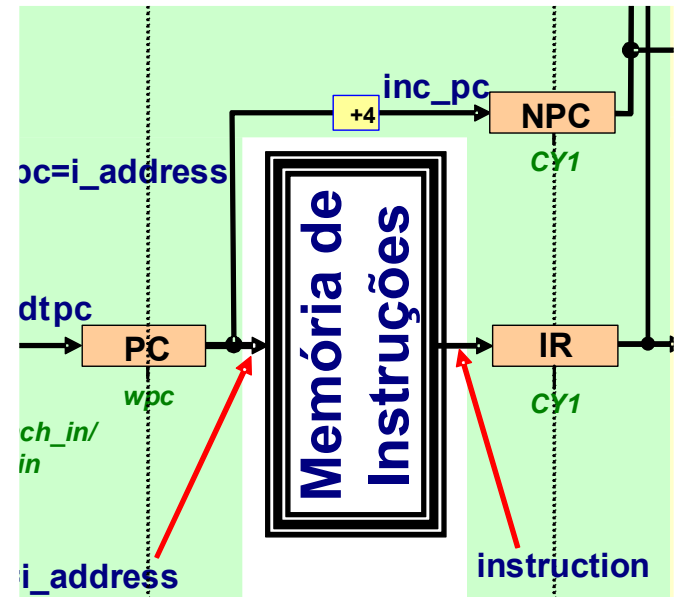
Organização: Registradores, Estruturas, Sinais de Controle

- Obs: Para facilitar a compreensão, abstrai-se as estruturas internas do Hw que dão suporte a MULTU/DIVU, da ALU, do Banco de Registradores etc.



Primeiro Estágio

- Usado por todas as instruções
 - Incrementa o PC
 - Armazena instrução no IR
 - NPC é registrador que guarda o PC após incrementar seu valor
 - Porquê existe o NPC?
 - Um motivo: não gerar transitórios em saltos
 - Ver especificação das instruções!



Segundo Estágio

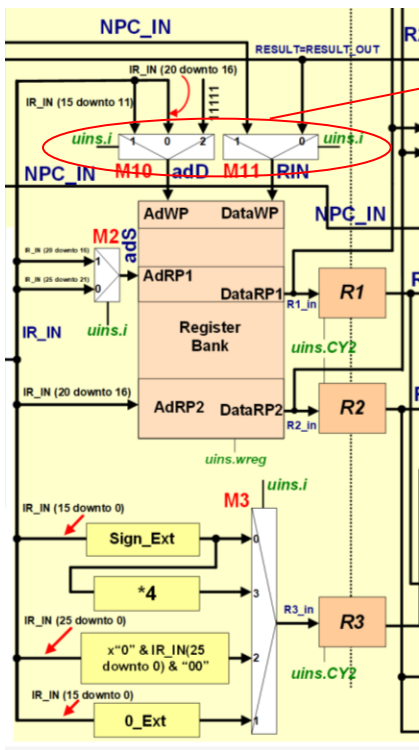
- Comum para todas as instruções (sinal CY2 ativado no segundo ciclo de toda instrução)
- Leitura do Banco de Registradores (sempre se lê 2 registradores, mesmo que 1 deles ou mesmo nenhum seja necessário à instrução)
- Determinação da constante imediata (sempre, mesmo que não seja usada na instrução)

Controles de escrita no Banco de Registradores

Serão discutidos no Quinto Estágio

Controles de escrita no Banco de Registradores

Serão discutidos no Quinto Estágio



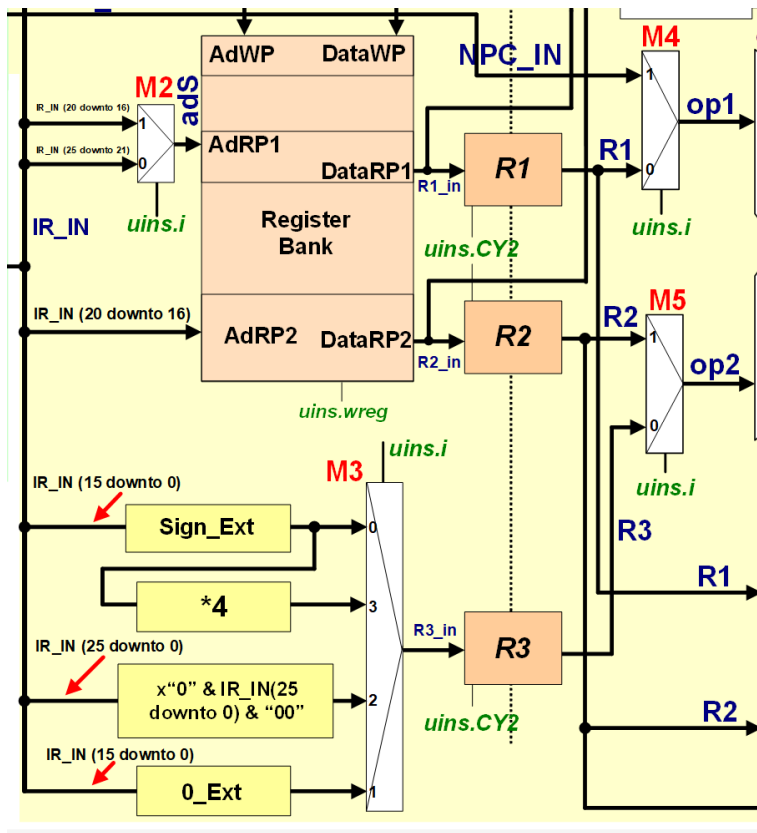
- Há 4 formas de gerar a constante imediata (mux **M3**)
 - Extensão de sinal (ADDIU, LW, LBU, SW, SB, SLTI, SLTIU e todas as instruções não mencionadas abaixo)
 - Extensão de sinal*4 (BEQ, BGEZ, BLEZ, BNE)
 - “0000” & IR(26 downto 0) & “00” (J, JAL)
 - Extensão de 0 (ANDI, ORI, XORI)

Segundo Estágio

- Controle do multiplexador **M2**

- Determina o endereço do registrador a ser lido do Banco e gravado em R1
- Trecho de código VHDL

```
ads <= IR_IN(20 downto 16) when uins.i=SSL or uins.i=SSRA or uins.i=SSRL else  
      IR_IN(25 downto 21);
```

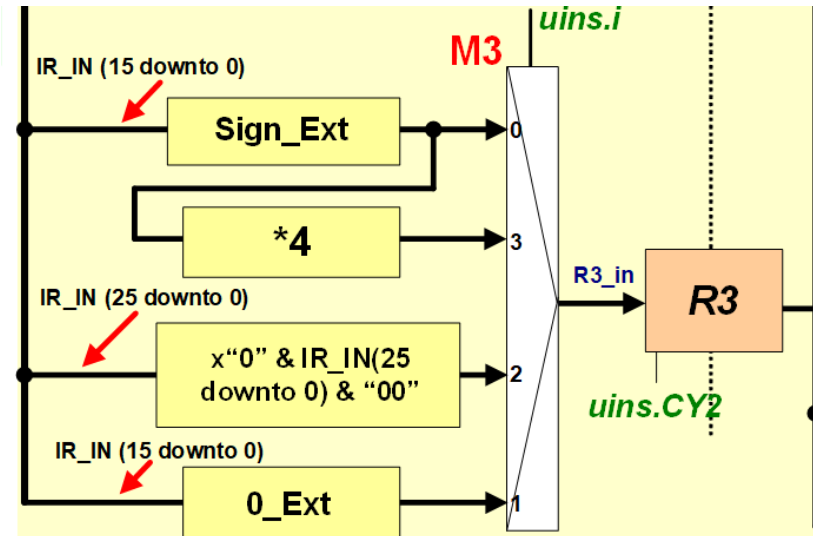


Porque **M2** é necessário?

- Devido às instruções de deslocamento (SLL, SRA e SRL)
- R2 e R3 compartilham uma entrada da ULA (ver Terceiro estágio)
- As instruções de deslocamento usam a constante para especificar a quantidade de bits a deslocar
- Logo, o registrador fonte, que é especificado nos bits (20 downto 16) destas instruções é lido e gravado em R1

Segundo Estágio

- **Multiplexador M3**
 - Determina o valor do dado imediato (constante)



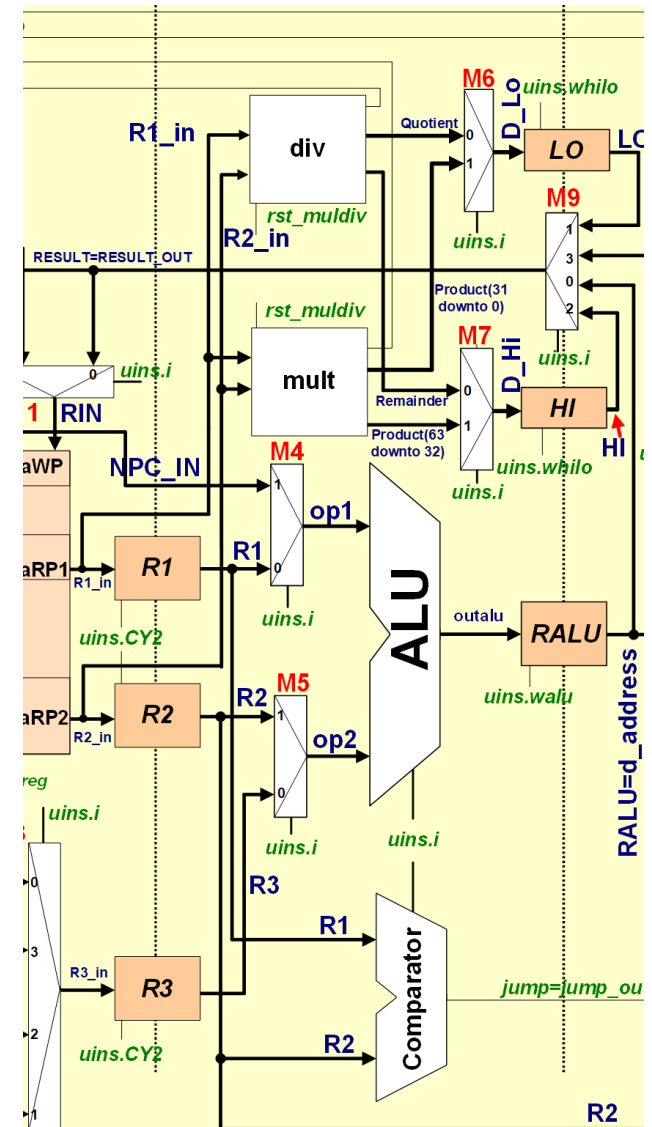
- **Trecho de código VHDL correspondente**

```
sign_extend <=  x"FFFF" & IR_IN(15 downto 0) when IR_IN(15)='1' else
                x"0000" & IR_IN(15 downto 0);
```

```
R3_in <= sign_extend(29 downto 0)    & "00" when inst_branch='1'           else -- ext de sinal*4
        "0000" & IR(25 downto 0)    & "00" when uins.i=J or uins.i=JAL else -- para J/JAL
        x"0000" & IR(15 downto 0)    when uins.i={ANDI,ORI,XORI} else      -- ext de zero
        sign_extend;                                                         -- ext de sinal
```

Terceiro Estágio

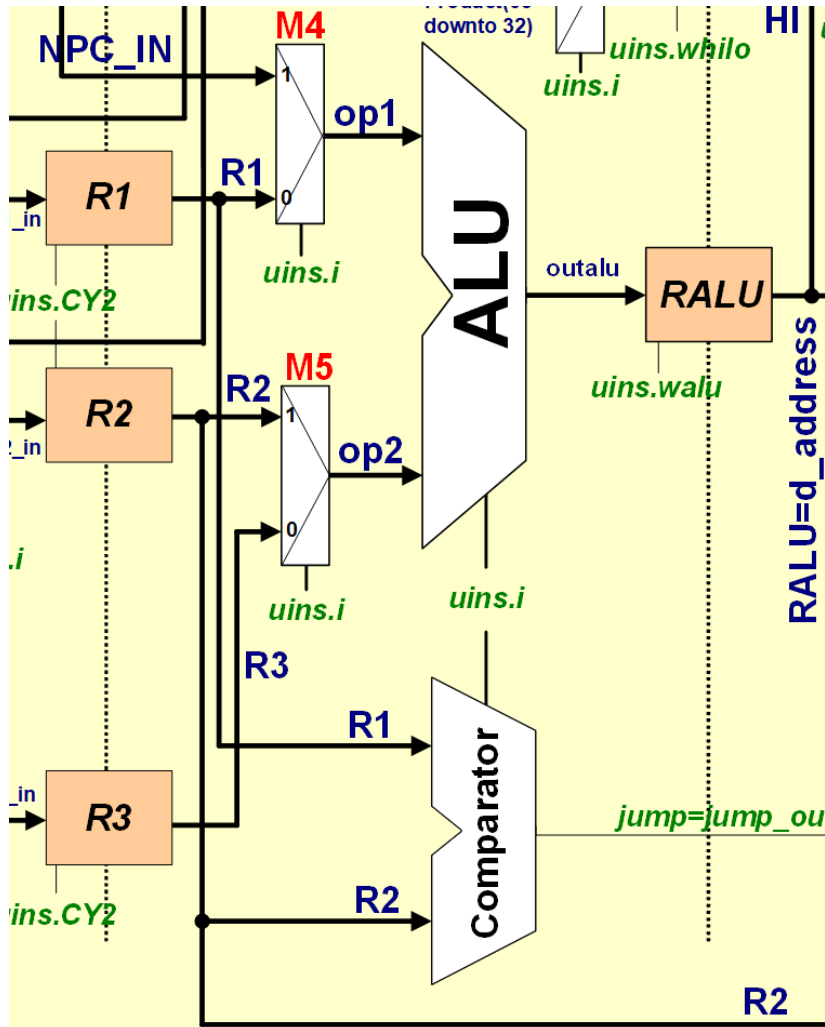
- **Estágio comumente utilizado pela maioria das instruções**
 - **ULA (ALU) → executa uma das 17 operações que é projetada para realizar**
 - Ver Tabela 3 da Especificação da MIPS_S para o detalhamento das 17 operações
 - As 17 operações da ULA servem a 33 das 37 instruções
 - As demais 4 instruções (MULTU, DIVU, MFHI e MFLO) não usam a ULA para nada
 - **Comparador → Determina um *flag* (denominado *jump*) que indica se a condição de uma instrução de salto é verdadeira ou não**
 - *jump* é usado apenas para as instruções de salto condicional (BEQ, BGEZ, BLEZ, BNE)
 - O módulo mult existe apenas para servir a instrução MULTU; ele recebe as saídas das portas de leitura do banco (DataRP1 e DataRP2), dois dados de 32bits realiza a multiplicação destes e gera a saída produto de 64 bits. A parte alta do resultado, **produto (63 downto 32)**, é armazenada no registrador Hi e a parte baixa, **produto (31 downto 0)**, é armazenada no registrador Lo. Também se gera um sinal (*end_mul* que quando em '1' indica que a multiplicação foi concluída.
 - O módulo div existe apenas para servir a instrução DIVU; ele recebe as saídas das portas de leitura do banco (DataRP1 e DataRP2), dois dados de 32bits realiza a divisão destes e gera duas saídas de 32 bits, o **quociente** e o **resto**. O primeiro é armazenado no registrador Lo e o último é armazenado no registrador Hi. Também se gera um sinal (*end_div* que quando em '1' indica que a multiplicação foi concluída.



Terceiro Estágio

- Controle do multiplexador **M4**

```
op1 <= npc when inst_branch='1' else R1;
```



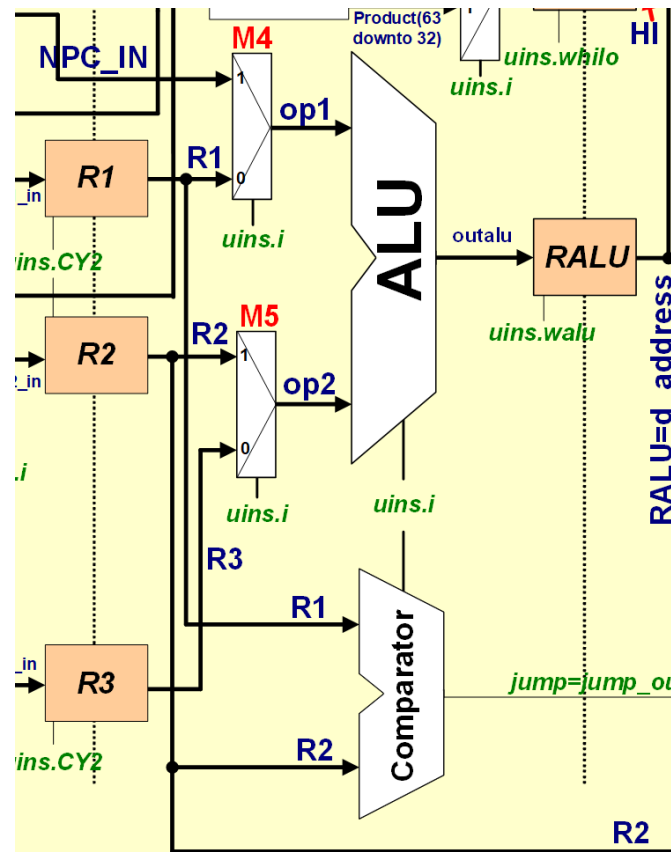
- Quando se tem uma instrução de salto condicional, a ULA determina o endereço do salto, somando o valor do NPC à constante imediata

Terceiro Estágio

- O Comparador

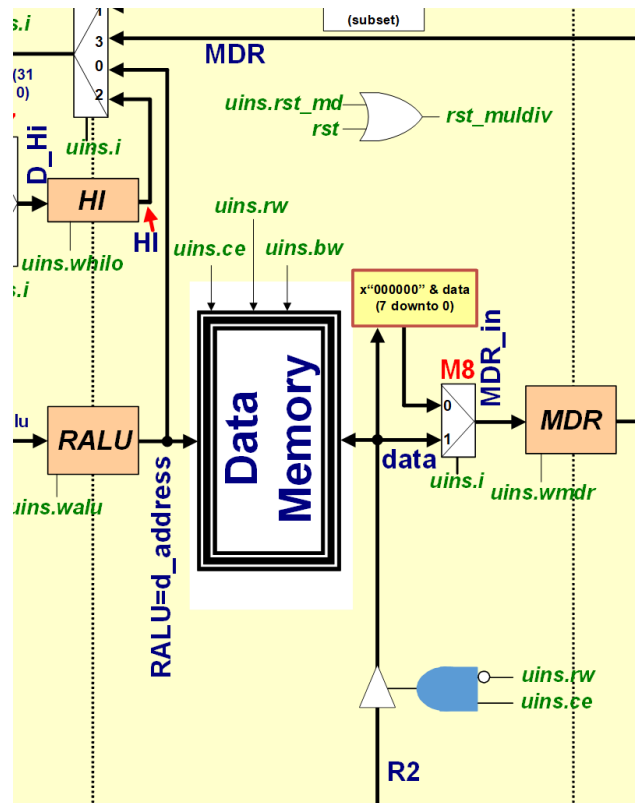
```
jump <= '1' when ( (R1=R2 and uins.i=BEQ) or (R1>=0 and uins.i=BGEZ) or  
                  (R1<=0 and uins.i=BLEZ) or (R1/=R2 and uins.i=BNE) ) else  
                  '0';
```

- O valor poderia ser armazenado em um flip-flop
- De fato, este valor é encaminhado para o Bloco de Controle, que decide o que carregar no PC no próximo ciclo



Quarto Estágio

- Utilizado apenas pelas instruções LBU/LW/SW/SB
- Leitura (LB/LW)
 - Valor endereçado pelo registrador RALU é lido da Memória de Dados e escrito no registrador MDR
 - Posição de da Memória de Dados endereçada pelo registrador RALU recebe o conteúdo do registrador RB quando for instrução de escrita

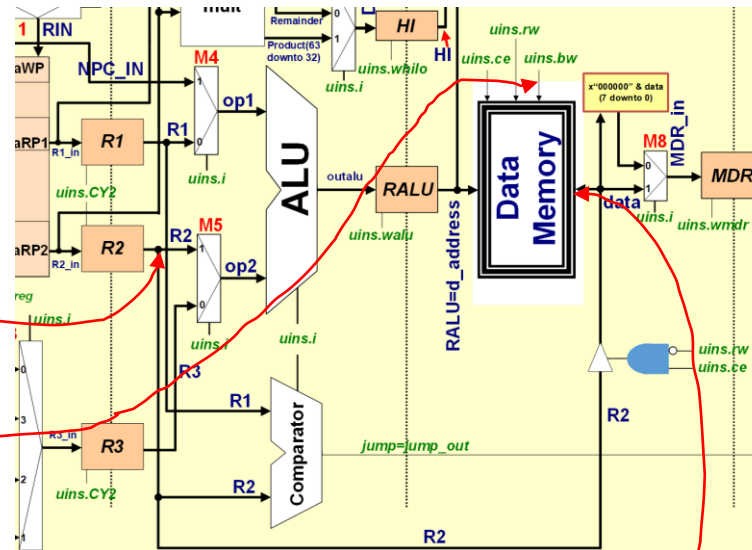
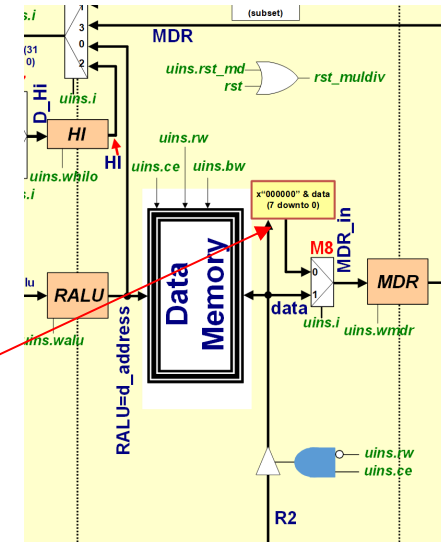


Quarto Estágio

- **Acesso a byte**

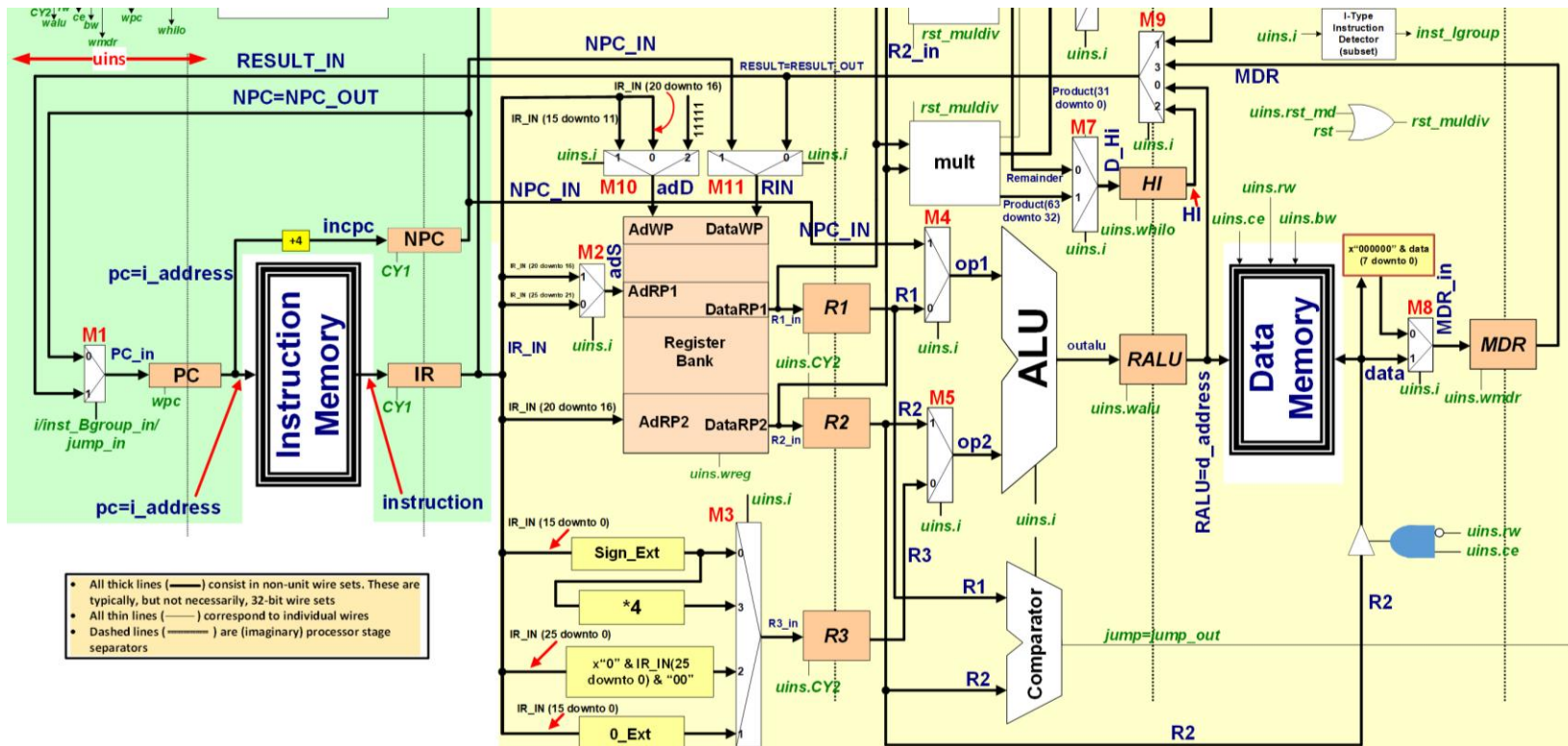
- **Load Byte Unsigned (LBU):** na instrução LBU, grava-se em MDR o byte menos significativo (LSB) e coloca-se em zero os 3 bytes mais significativos (MSBs com x“000000”)

- **Store Byte (instrução SB):** a memória recebe 32 bits (provenientes do Registrador auxiliar R2) e um sinal de controle (bw=0) para selecionar escrever apenas o byte menos significativo



Quinto Estágio

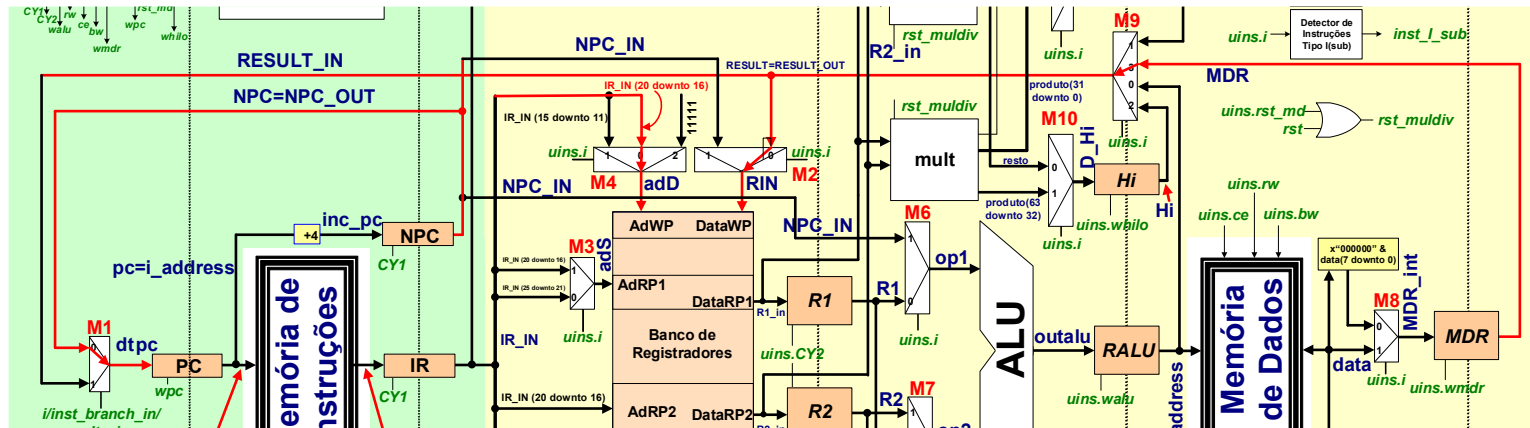
- Estágio que conclui a execução de uma determinada instrução
- Denomina-se em inglês *write-back* (pois, na maioria das instruções escreve de volta no banco de registradores o resultado da instrução)
- Depende da instrução corrente



Quinto Estágio

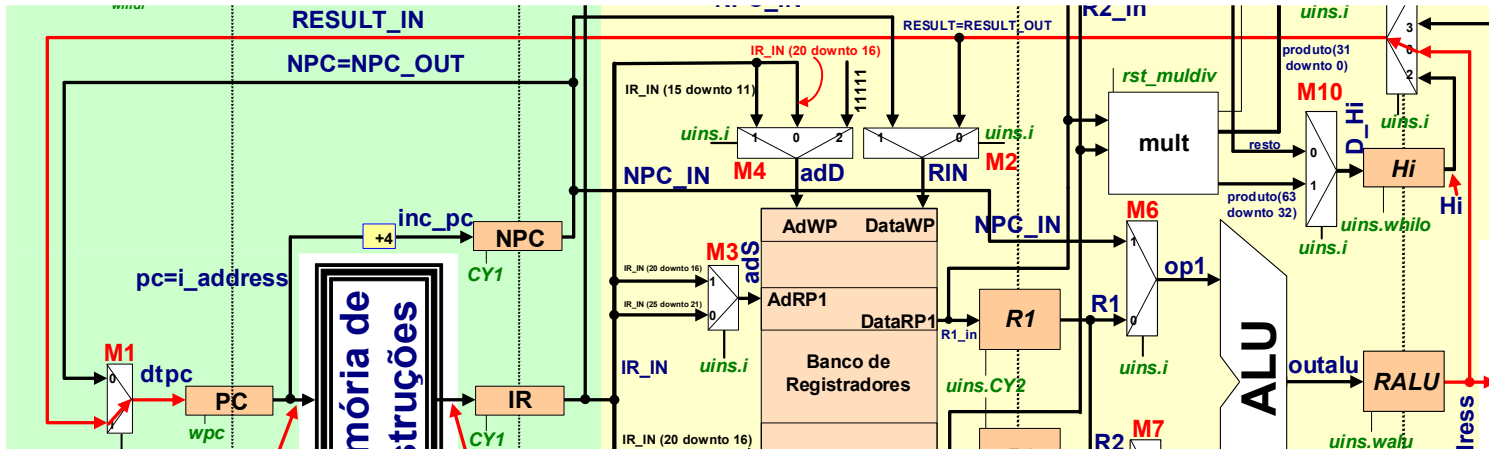
- Instruções LW/LB

- Valor armazenado em MDR é escrito no banco de registradores
- O valor do NPC (PC incrementado de 4) é escrito no PC



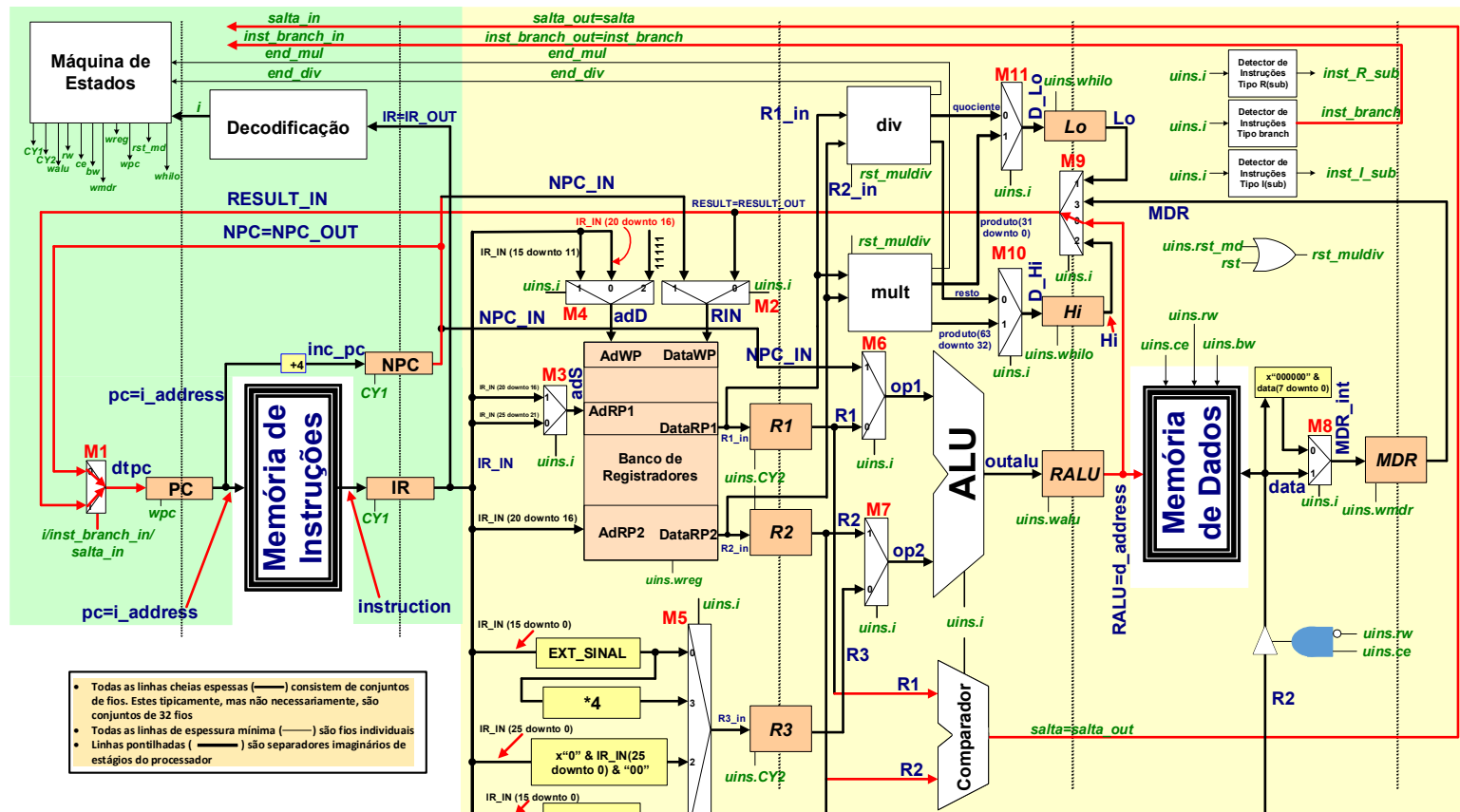
Quinto Estágio

- **Instruções de salto incondicional**
 - **J / JR**
 - **PC recebe valor contido no registrador RALU**



Quinto Estágio

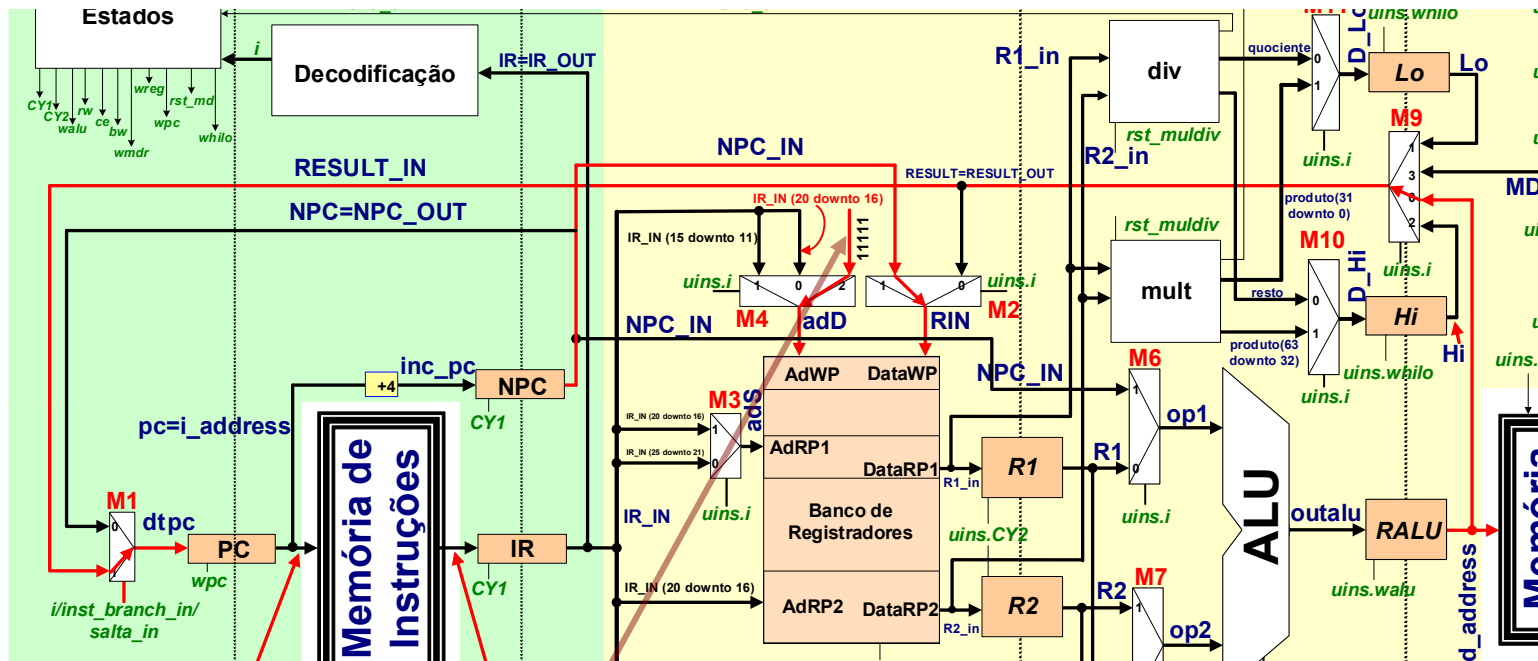
- Instruções de salto condicional
 - O PC pode receber o NPC ou o endereço destino, calculado na ULA
 - O controle do mux M1 é a saída do comparador



Quinto Estágio

- **Instrução JAL (Jump and Link)**

- O PC recebe o endereço da rotina
- O registrador \$RA (endereço “11111” recebe o NPC (PC+4, o endereço de retorno da subrotina)



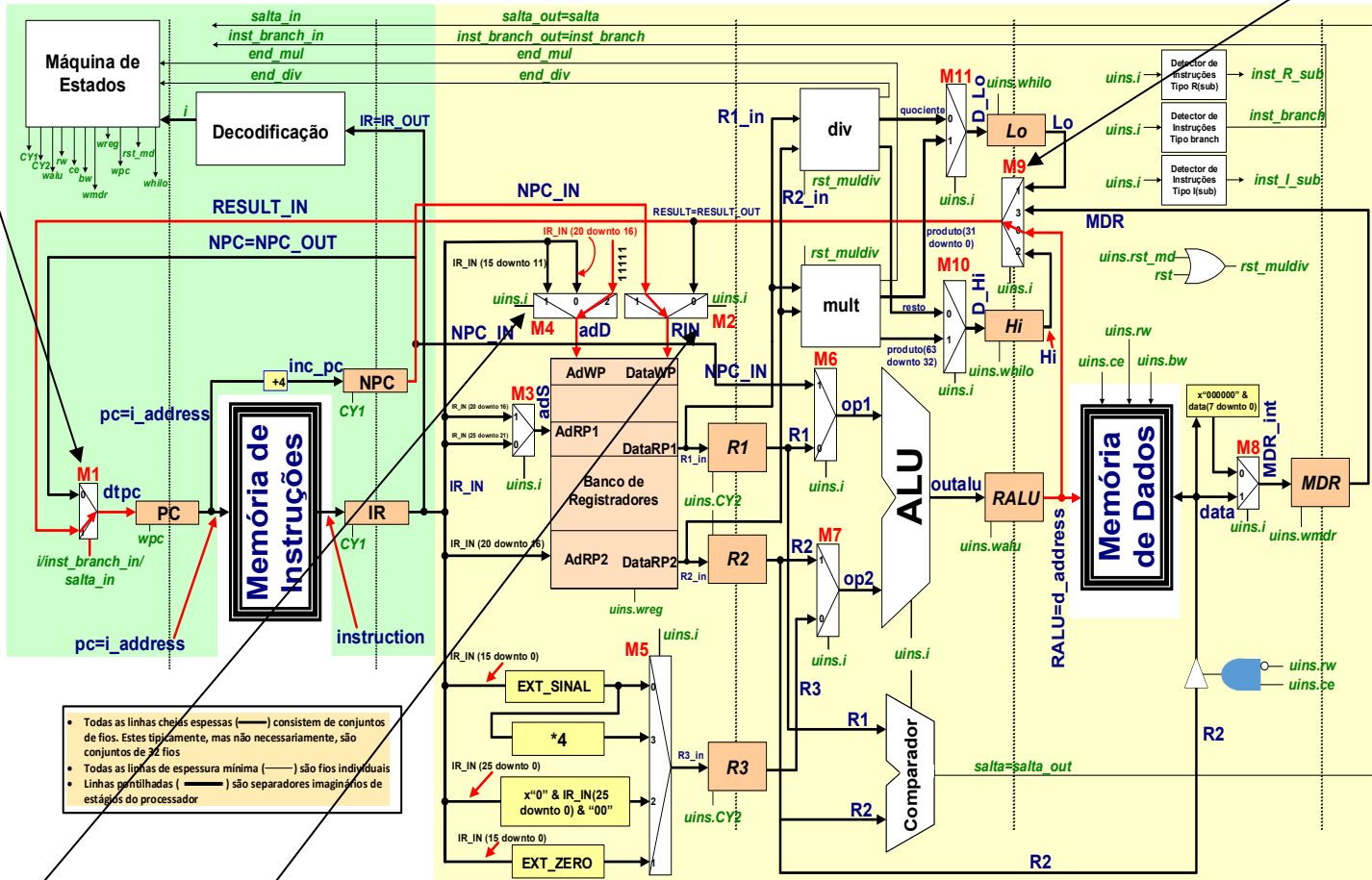
ENDEREÇO 31 (registrador \$RA)

Quinto Estágio

Alguns dos multiplexadores do quinto estágio

```
result <= MDR when i={LW,LBU} else RALU;
```

```
dtpc <= result when (inst_branch='1' and salta='1') or i={J,JAL,JALR,JR} else npc;
```

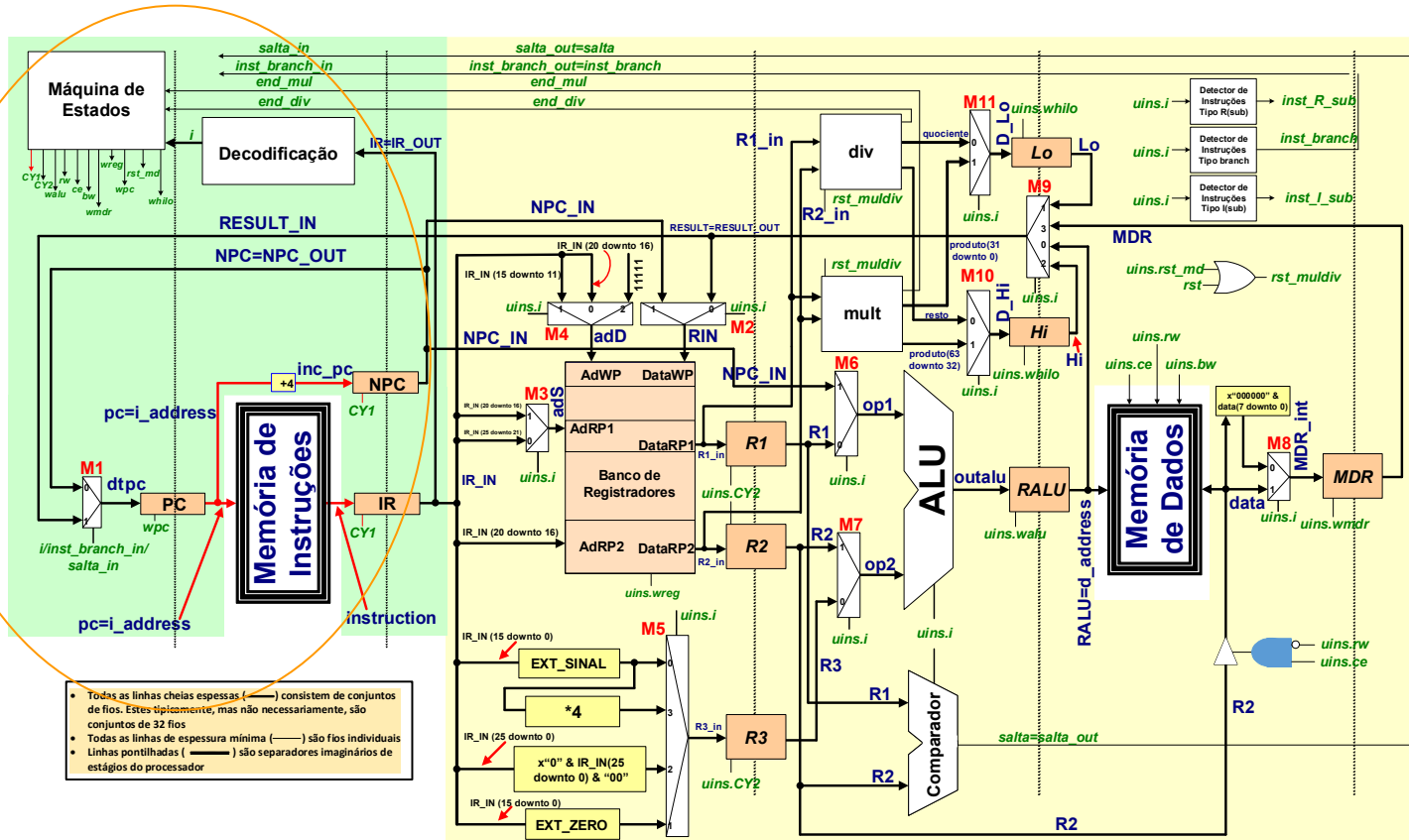


```
RIN <= npc when (uins.i=JALR or uins.i=JAL) else result;
```

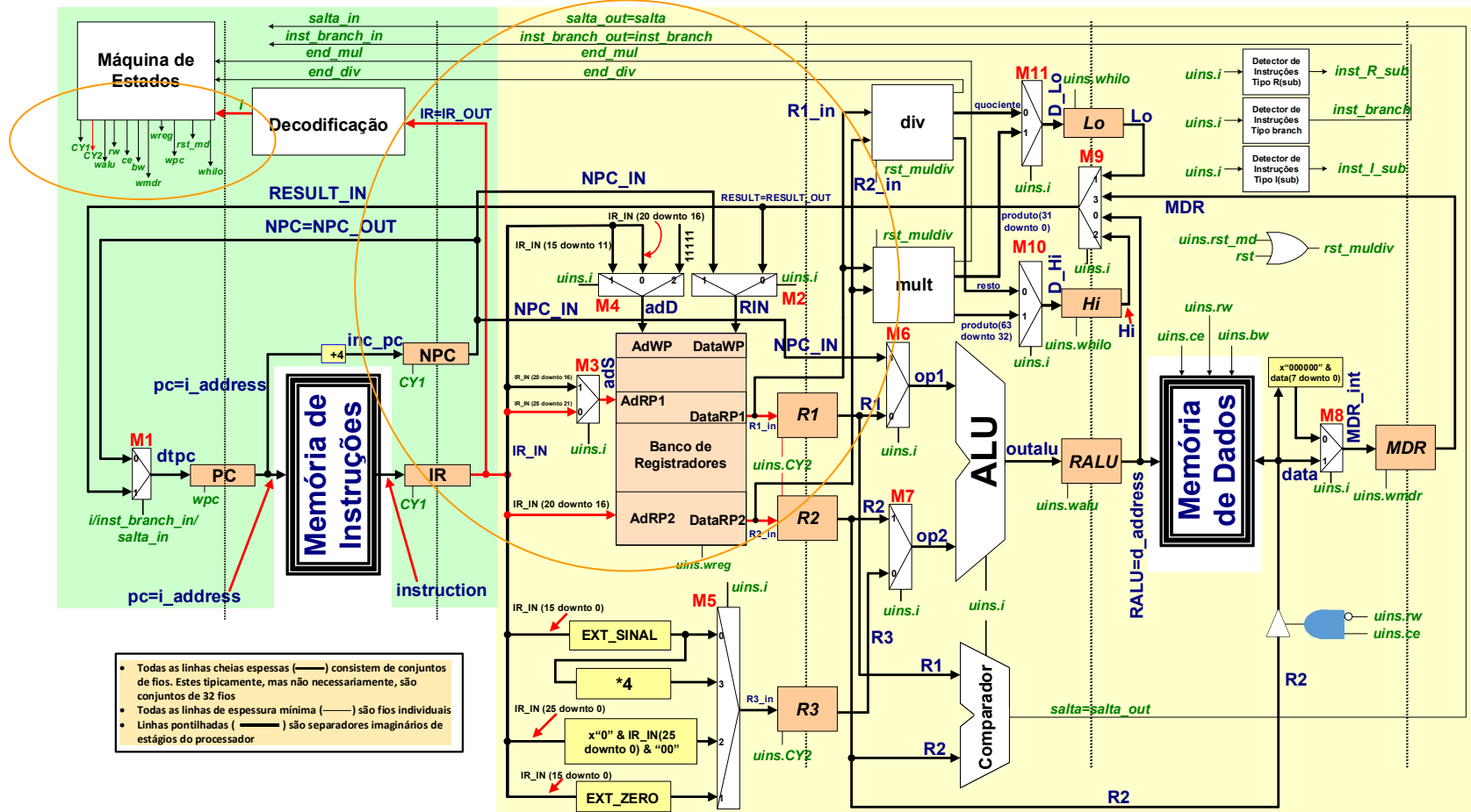
```
add <= 31 when uins.i=JAL else
  IR(15 downto 11) when i={ADDU, SUBU, AND, OR, XOR, NOR, SLTU, SLT, JALR, desloc} else
  IR(20 downto 16);
```

Execução ciclo de clock a ciclo de clock (Instruções Lógicas e Aritméticas, tipo R)

- Primeiro ciclo: busca da operação (fetch)

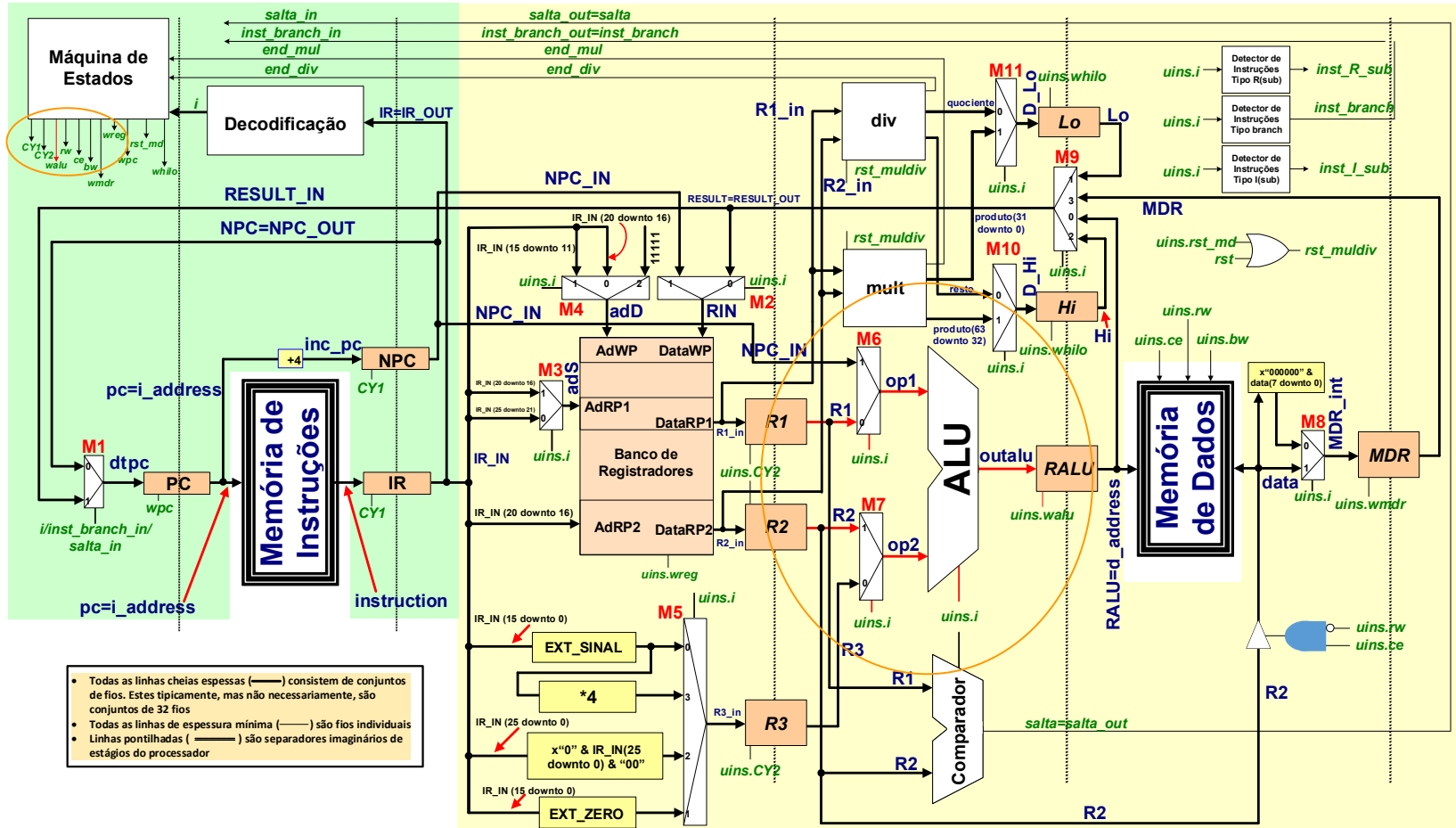


- **Decodificação da instrução e leitura dos registradores fonte**



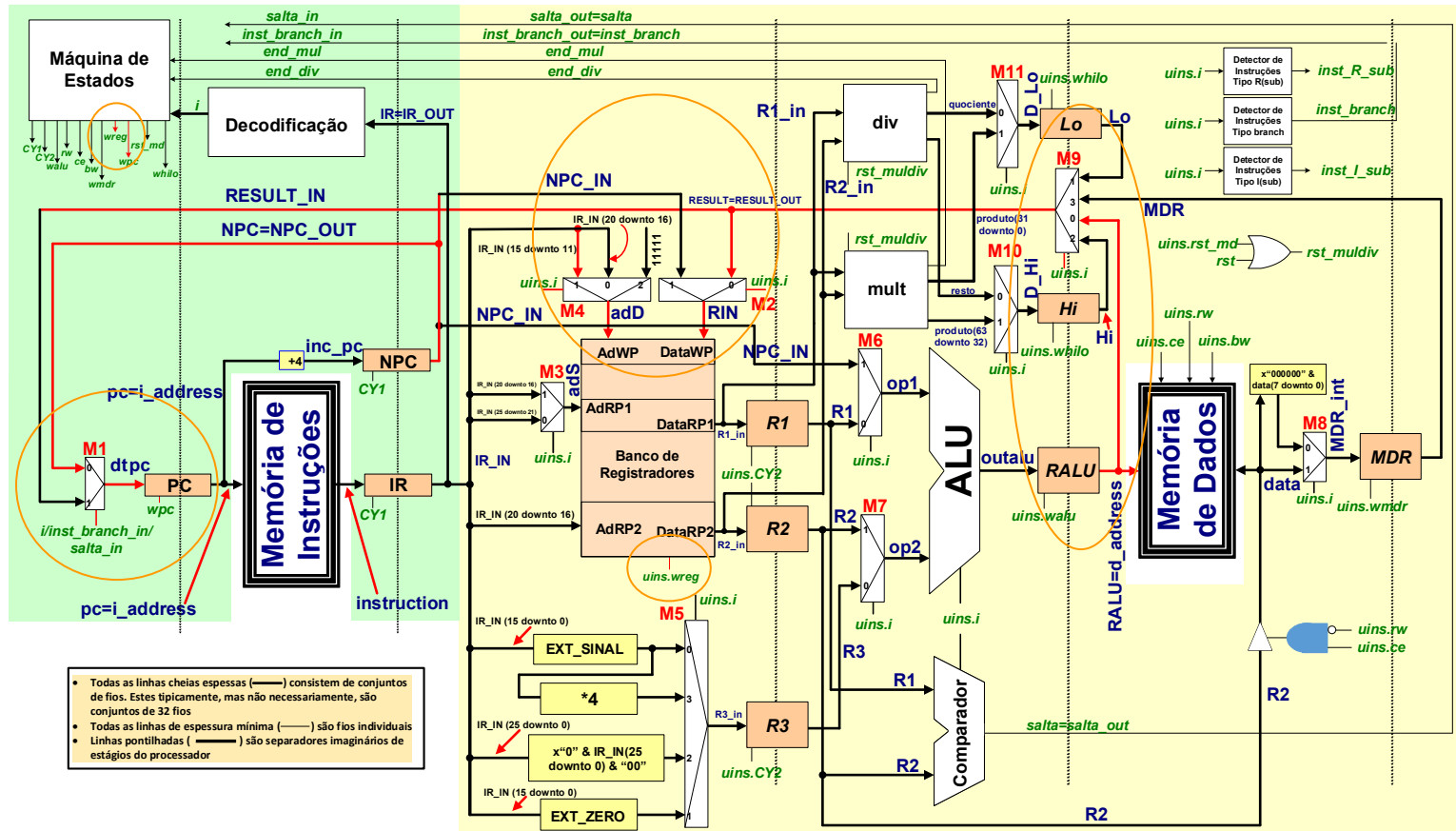
Execução ciclo de clock a ciclo de clock (Instruções Lógicas e Aritméticas, tipo R)

- Terceiro ciclo: operação com a ULA



Execução ciclo de clock a ciclo de clock (Instruções Lógicas e Aritméticas, tipo R)

- Quarto ciclo: atualiza PC e escreve no banco de registradores



ULA – Código VHDL Parcial

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
use IEEE.std_logic_arith.all;
use work.p_MIPS_S.all;
```

Atenção: operações sem sinal por *default*

```
entity alu is
    port( op1, op2 : in reg32;
          outalu : out reg32;
          op_alu : in inst_type
    );
end alu;
```

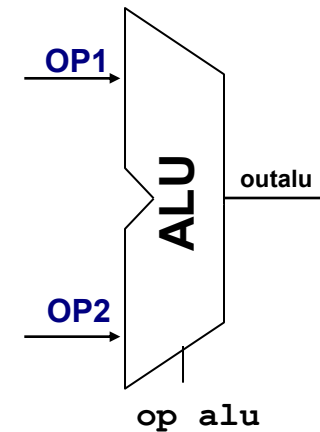
```
architecture alu of alu is
    signal menorU, menorS : std_logic ;
begin
```

```
    menorU <= '1' when op1 < op2 else '0';
```

COMPARAÇÃO sem sinal

```
    menorS <= '1' when ieee.std_logic_signed."<"(op1, op2) else '0' ; -- signed
```

COMPARAÇÃO com sinal – sobrecarga manual do operador <



- Observar uso do tipo *inst_type* na entity

ULA – Código VHDL Parcial

- Observe
 - uso de concatenação nas instruções SLTxx
 - operações de deslocamento (conversões para poder usar operadores de deslocamento)

```
outalu <= op1 - op2          when op_alu=SUBU          else
        op1 and op2        when op_alu=AAND or op_alu=ANDI else
        op1 or op2         when op_alu=OOR or op_alu=ORI  else
        op1 xor op2        when op_alu=XXOR or op_alu=XORI else
        op2(15 downto 0) & x"0000"                    when op_alu=LUI          else
        (0=>menorU, others=>'0')                        when op_alu=SLTU or op_alu=SLTIU else
        (0=>menorS, others=>'0')                        when op_alu=SLT or op_alu=SLTI  else
        op1(31 downto 28) & op2(27 downto 0)            when op_alu=J          else
        op1              when op_alu=JR or op_alu=JALR   else
        to_StdLogicVector(to_bitvector(op1) sll CONV_INTEGER(op2(10 downto 6)))
                           when op_alu=SSLL              else
        to_StdLogicVector(to_bitvector(op2) sll CONV_INTEGER(op1(5 downto 0)))
                           when op_alu=SLLV              else
        to_StdLogicVector(to_bitvector(op1) sra CONV_INTEGER(op2(10 downto 6)))
                           when op_alu=SSRA              else
        to_StdLogicVector(to_bitvector(op2) sra CONV_INTEGER(op1(5 downto 0)))
                           when op_alu=SRAV              else
        to_StdLogicVector(to_bitvector(op1) srl CONV_INTEGER(op2(10 downto 6)))
                           when op_alu=SSRL              else
        to_StdLogicVector(to_bitvector(op2) srl CONV_INTEGER(op1(5 downto 0)))
                           when op_alu=SRLV              else
        op1 + op2;          -- default é fazer soma!!

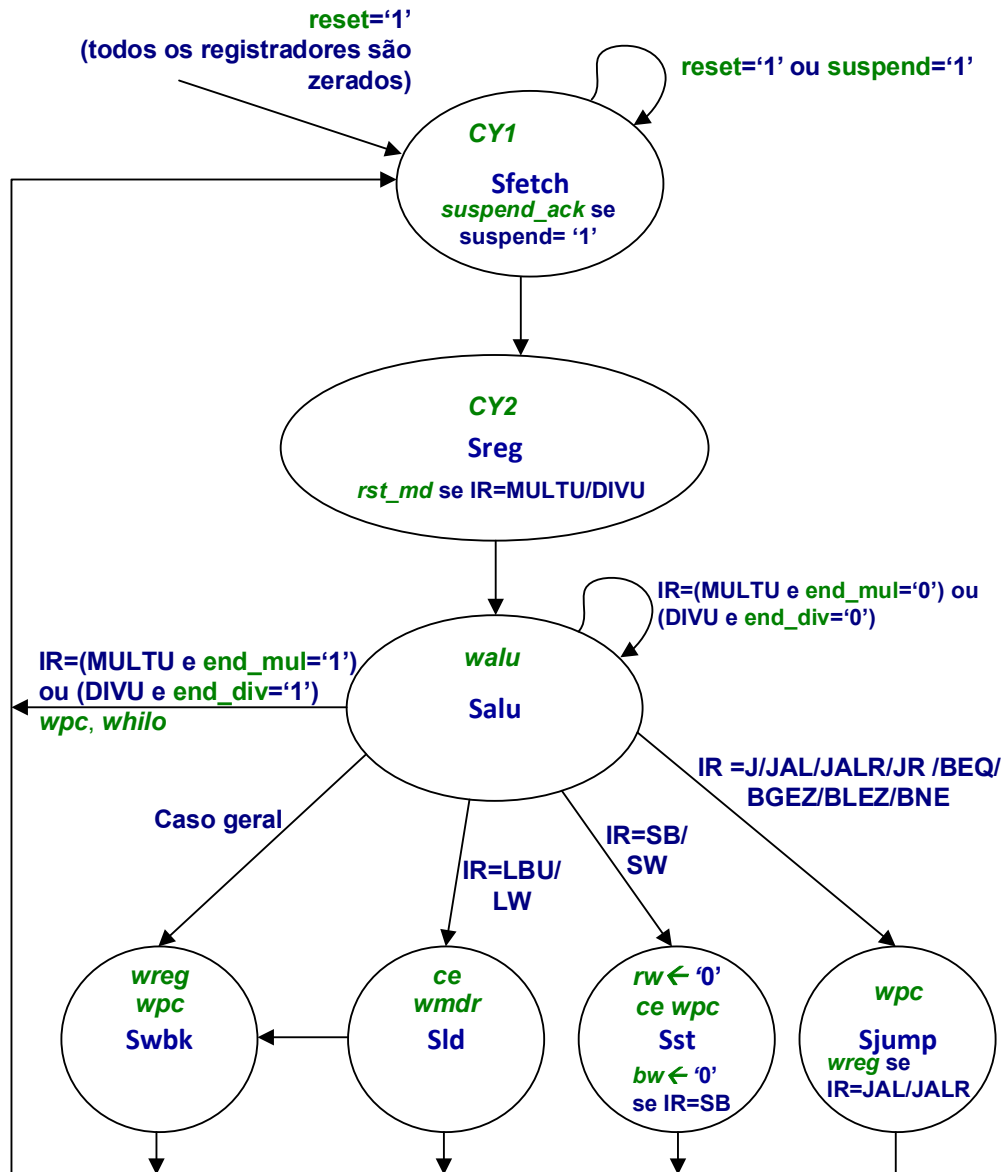
end alu;
```

Bloco de controle do Processador MIPS_S

- Microinstrução (para guardar todos os sinais de controle produzidos no Bloco de Controle) → definida no package p_MIPS_S

```
type microinstruction is record
  CY1:    std_logic;      -- write enable for regs of 1st stage
  CY2:    std_logic;      --      "  enable for regs of 2nd stage
  walu:   std_logic;      --      "  enable for third stage reg
  wmdr:   std_logic;      --      "  enable for fourth stage reg
  wpc:    std_logic;      -- PC write enable
  wreg:   std_logic;      -- Register Bank write enable
  whilo:  std_logic;      -- HI and LO registers write enable
  ce:     std_logic;      -- Data Memory CE and R/W controls
  rw:     std_logic;
  bw:     std_logic;      -- Data Memory Byte/Word write control
  i:      inst_type;      -- instruction identification
  rst_md: std_logic;
end record;
```

Bloco de controle do Processador MIPS_S



A principal máquina de estados de controle do processador

- Os 3 primeiros ciclos são os mesmo para todas as instruções, mas não o que é realizado neles, pelo menos no segundo e terceiro há variações

Entidade/Arquitetura do Bloco de Controle

```
library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Std_Logic_unsigned.all;
use work.p_MIPS_S.all;

entity control_unit is
    port( ck, rst, suspend : in std_logic;
          suspend_ack : out std_logic;
          i_address : out wires32;
          instruction : in wires32;
          IR_OUT : out wires32;
          uins : out microinstruction;
          NPC_OUT : out wires32;
          inst_Bgroup_in, jump_in : in std_logic;
          end_mul, end_div : in std_logic;
          RESULT_IN : in wires32;
    );
end control_unit;

architecture control_unit of control_unit is
    type type_state is (Sfetch, Sreg, Salu, Swbk, Sld, Sst, Ssalta);
    signal PS, NS : type_state;
    signal i : inst_type;
    signal uins_int : microinstruction;
    signal PC_in, PC, incpc, NPC, IR : wires32;
    signal suspend_ack_intcu : std_logic;

begin
```

Tipo enumerado para a máquina de estados

Primeira parte – decodificação de instruções

```
i <=  ADDU    when IR(31 downto 26)="000000" and IR(10 downto 0)="00000100001" else
      SUBU    when IR(31 downto 26)="000000" and IR(10 downto 0)="00000100011" else
      AAND    when IR(31 downto 26)="000000" and IR(10 downto 0)="00000100100" else
      OOR     when IR(31 downto 26)="000000" and IR(10 downto 0)="00000100101" else
      XXOR    when IR(31 downto 26)="000000" and IR(10 downto 0)="00000100110" else
      NNOR    when IR(31 downto 26)="000000" and IR(10 downto 0)="00000100111" else
      SLL     when IR(31 downto 21)="00000000000" and IR(5 downto 0)="000000" else
      SLLV    when IR(31 downto 26)="000000" and IR(10 downto 0)="00000000100" else
      SSRA    when IR(31 downto 21)="00000000000" and IR(5 downto 0)="000011" else
      SRAV    when IR(31 downto 26)="000000" and IR(10 downto 0)="00000000111" else
      SSRL    when IR(31 downto 21)="00000000000" and IR(5 downto 0)="000010" else
      SRLV    when IR(31 downto 26)="000000" and IR(10 downto 0)="00000000110" else
      ADDIU   when IR(31 downto 26)="001001" else
      ANDI    when IR(31 downto 26)="001100" else
      ....
      JR      when IR(31 downto 26)="000000" and IR(20 downto 0)="000000000000000001000" else
      MULTU   when IR(31 downto 26)="000000" and IR(15 downto 0)="0000000000011001" else
      DIVU    when IR(31 downto 26)="000000" and IR(15 downto 0)="0000000000011011" else
      MFHI    when IR(31 downto 16)=x"0000" and IR(10 downto 0)="00000010000" else
      MFLO    when IR(31 downto 16)=x"0000" and IR(10 downto 0)="00000010010" else
      invalid_instruction ;
      -- IMPORTANT: the default condition is invalid instruction
assert i /= invalid_instruction
      report "***** INVALID INSTRUCTION *****"
      severity error;

uins_int.i <= i;
```

COMPONENTE PRINCIPAL DA MICROINSTRUÇÃO

Segunda parte – escrita nos registradores / acesso memória

```
uins_int.CY1 <= '1' when PS=Sfetch else '0';  
uins_int.CY2 <= '1' when PS=Sreg else '0';
```

```
uins_int.rst_md <= '1' when PS=Sreg and  
    (i=MULTU or i=DIVU) else '0';  
uins_int.walu <= '1' when PS=Salu else '0';
```

```
uins_int.ce <= '1' when PS=Sld or PS=Sst  
    else '0';
```

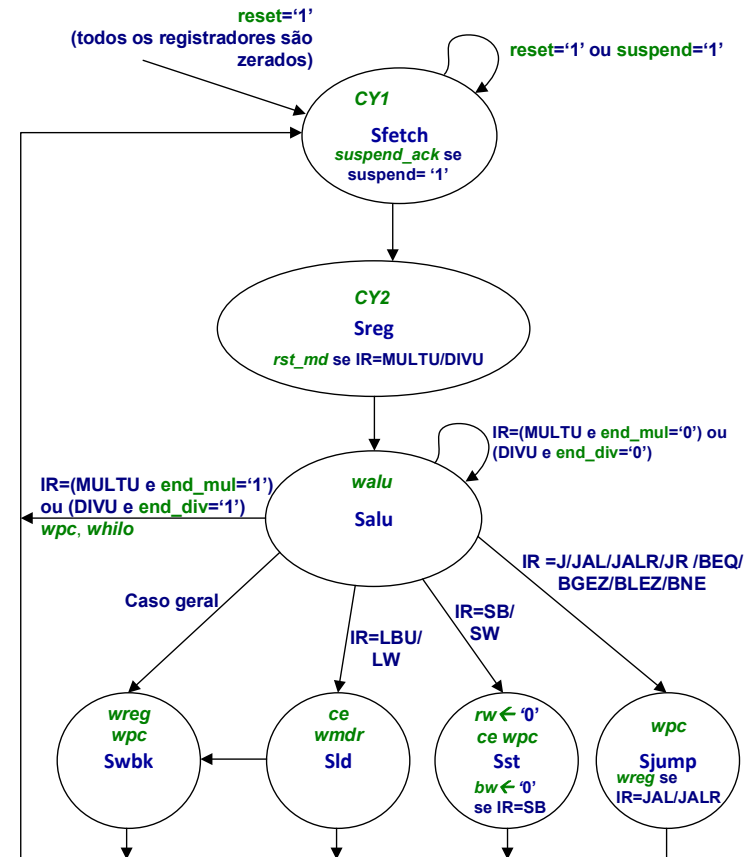
```
uins_int.rw <= '0' when PS=Sst else '1';  
uins_int.bw <= '1' when (PS=Sst and i=SB)  
    else '0';
```

```
uins_int.wmdr <= '1' when PS=Sld else '0';
```

```
uins_int.wreg <= '1' when PS=Swbk or (PS=Ssalta  
    and (i=JALR or i=JAL)) else '0';
```

```
uins_int.wpc <= '1' when PS=Swbk or PS=Sst or  
    PS=Ssalta else '0';
```

```
uins_int.whilo <= '1' when PS=Salu and  
    ((end_mul='1' and i=MULTU) or (end_div='1'  
    and i=DIVU)) else '0';
```



```
entity fsm is port(X, rst, ck : in std_logic;
                  Z: out std_logic);
```

```
end;
```

```
architecture A of fsm is
```

```
    type STATES is (S0, S1, S2, S3);
```

```
    signal scurrent, snext : STATES;
```

```
begin
```

```
    process(rst, ck)
```

```
    begin
```

```
        if rst='1' then scurrent <= S0;
```

```
        elsif ck'event and ck='1' then scurrent <= snext;
```

```
        end if;
```

```
    end process;
```

```
    process(scurrent, X)
```

```
    begin
```

```
        case scurrent is
```

```
            when S0 => if X='0' then Z<='0'; snext <= S1;
```

```
                       else Z<='1'; snext <= S2;
```

```
            end if;
```

```
            when S1 => if X='0' then Z<='1'; snext <= S2;
```

```
                       else Z<='0'; snext <= S1;
```

```
            end if;
```

```
            when S2 => if X='0' then Z<='1'; snext <= S2;
```

```
                       else Z<='0'; snext <= S3;
```

```
            end if;
```

```
            when S3 => if X='0' then Z<='0'; snext <= S0;
```

```
                       else Z<='0'; snext <= S1;
```

```
            end if;
```

```
        end case;
```

```
    end process;
```

```
end A;
```

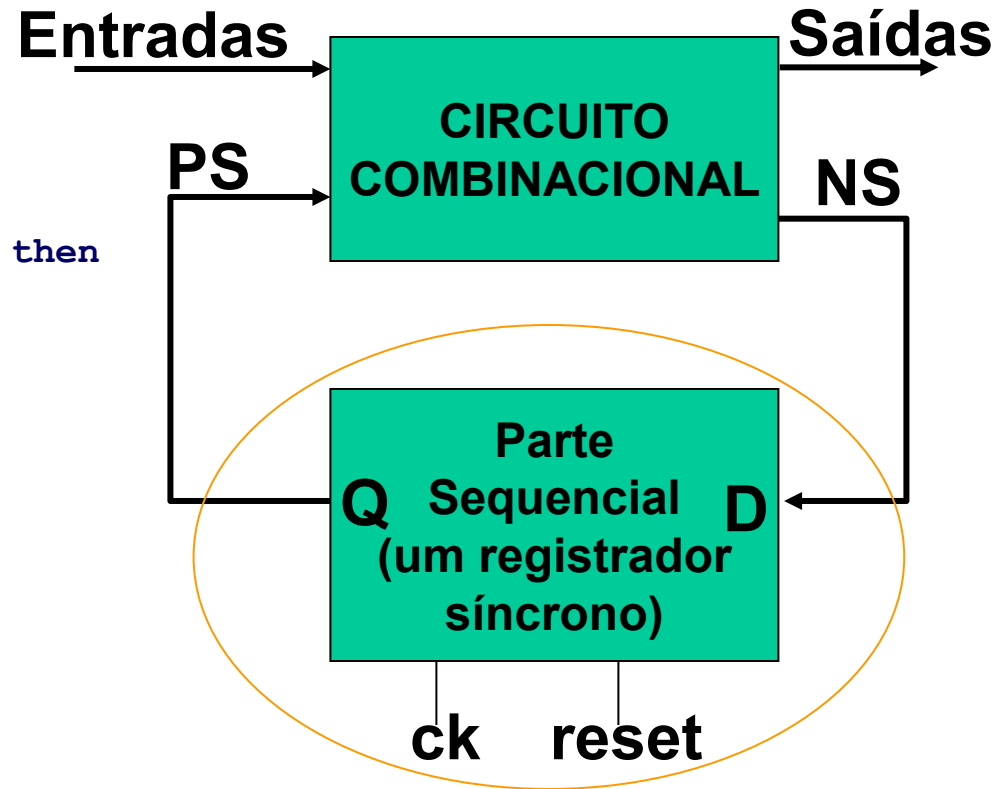
Exercício - Relembrando Máquinas de Estados

1. Desenhe o diagrama de transição de estados da máquina de estados finitos ao lado
2. Trata-se de uma máquina de Moore ou Mealy?
3. Justifique

Terceira parte – máquina de estados (1/2)

- Parte sequencial

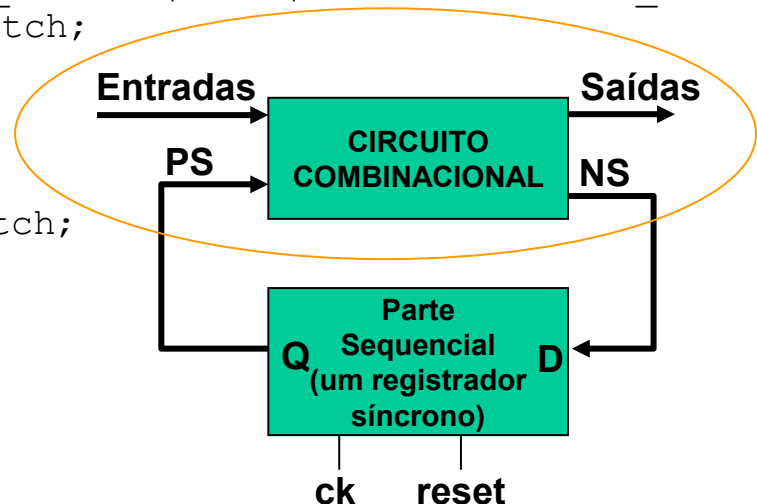
```
process(rst, ck)
begin
  if rst='1' then
    PS <= Sfetch;
  elsif ck'event and ck='1' then
    PS <= NS;
  end if;
end process;
```



Terceira parte – máquina de estados (2/2)

- Parte combinacional

```
process(PS, i, end_mul, end_div, suspend, suspend_ack_intcu)
begin
  case PS is
    when Sfetch => if (suspend='1') then NS <= Sfetch; else NS <= Sreg; end if;
    when Sreg => NS <= Salu;
    when Salu => if (i=LBU or i=LW) then NS <= Sld;
                  elsif (i=SB or i=SW) then NS <= Sst;
                  elsif ( i=J or i=JAL or i=JALR or i=JR or i=BEQ or i=BGEZ or
                          i=BLEZ or i=BNE) then NS <= Sjump;
                  elsif ((i=DIVU and end_div='0') or (i=MULTU and end_mul='0'))
                        then NS <= Salu;
                  elsif ((i=DIVU and end_div='1') or (i=MULTU and end_mul='1'))
                        then NS <= Sfetch;
                  else NS <= Swbk;
                  end if;
    when Sld => NS <= Swbk;
    when Sst | Sjump | Swbk => NS <= Sfetch;
  end case;
end process;
```



Tempo de execução de programas

- Calcule o tempo de execução para o programa abaixo, supondo clock de 50 MHz para a organização MIPS_S

```
main:    la      $t0,array
         la      $t1,size
         lw      $t1,0($t1)
         la      $t2,const
         lw      $t2,0($t2)

loop:    blez    $t1,end
         lw      $t3,0($t0)
         addu    $t3,$t3,$t2
         sw      $t3,0($t0)
         addiu   $t0,$t0,4
         addiu   $t1,$t1,-1
         j       loop

end:     j       end

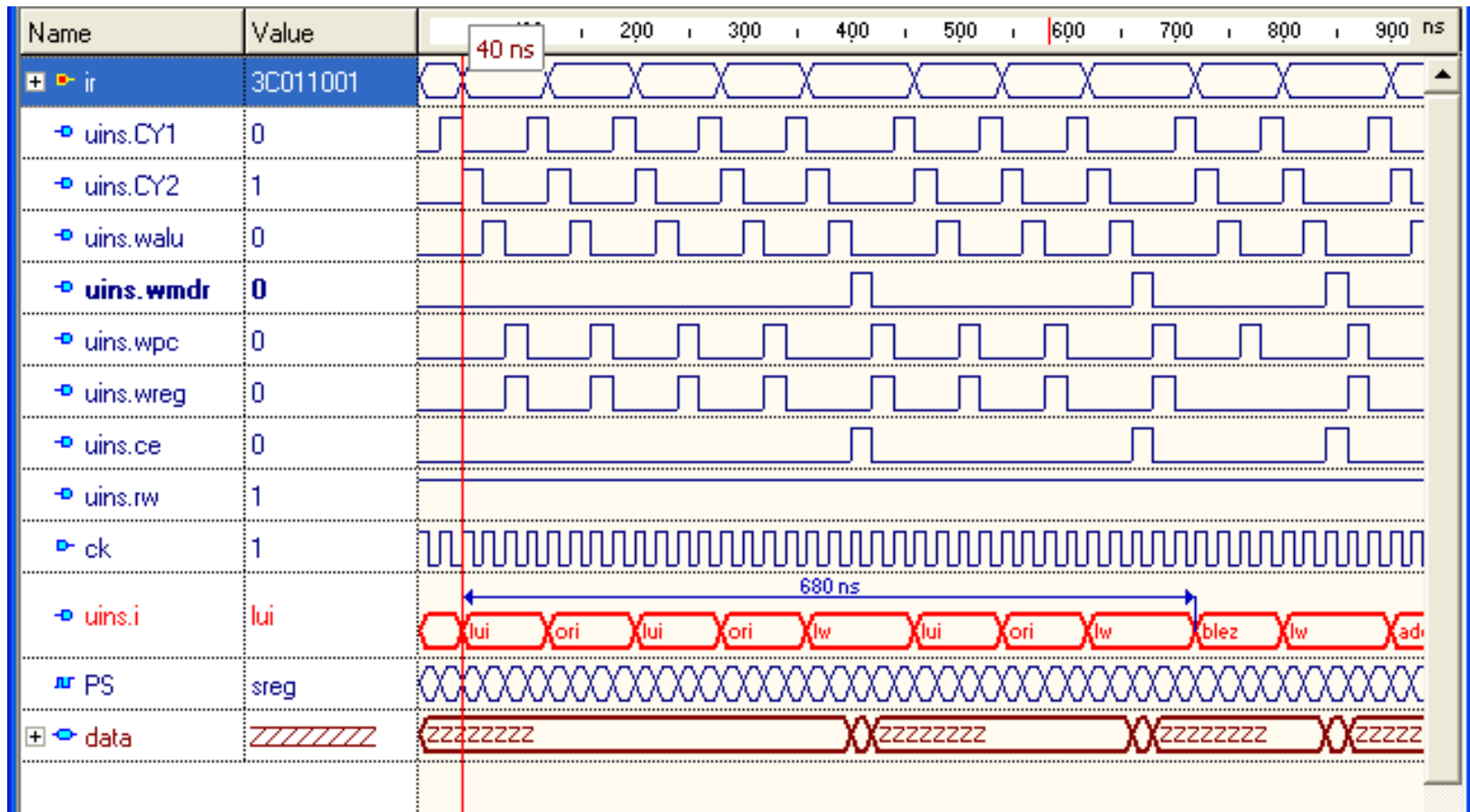
        .data
array:   .word   0x12 0xff 0x3 0x14 0x878 0x31 0x62 0x10 0x5 0x16 0x20
size:    .word   11
const:   .word   0x100
```

Resposta:
7,2 microsegundos

Fase de inicialização das variáveis

- 680 ns → 34 ciclos de clock
- Análise a microinstrução e suas partes

```
la    $t0,array
la    $t1,size
lw    $t1,0($t1)
la    $t2,const
lw    $t2,0($t2)
```

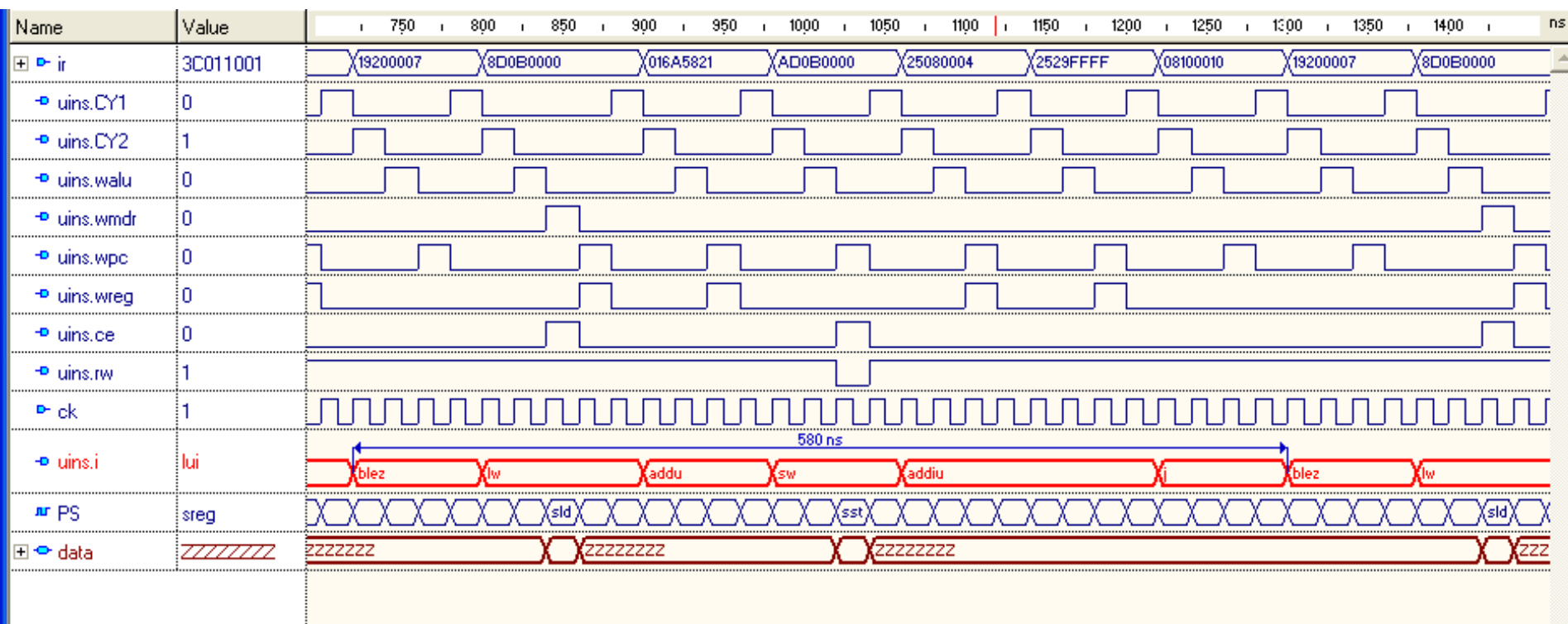


Uma iteração do laço

- 580 ns → 29 ciclos de clock

```

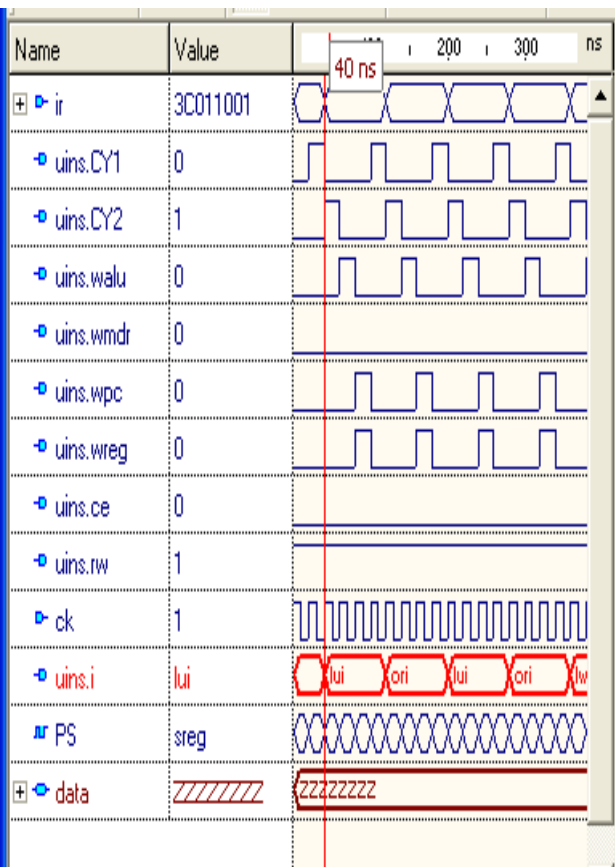
loop: blez  $t1,end
      lw    $t3,0($t0)
      addu  $t3,$t3,$t2
      sw    $t3,0($t0)
      addiu $t0,$t0,4
      addiu $t1,$t1,-1
      j     loop
    
```



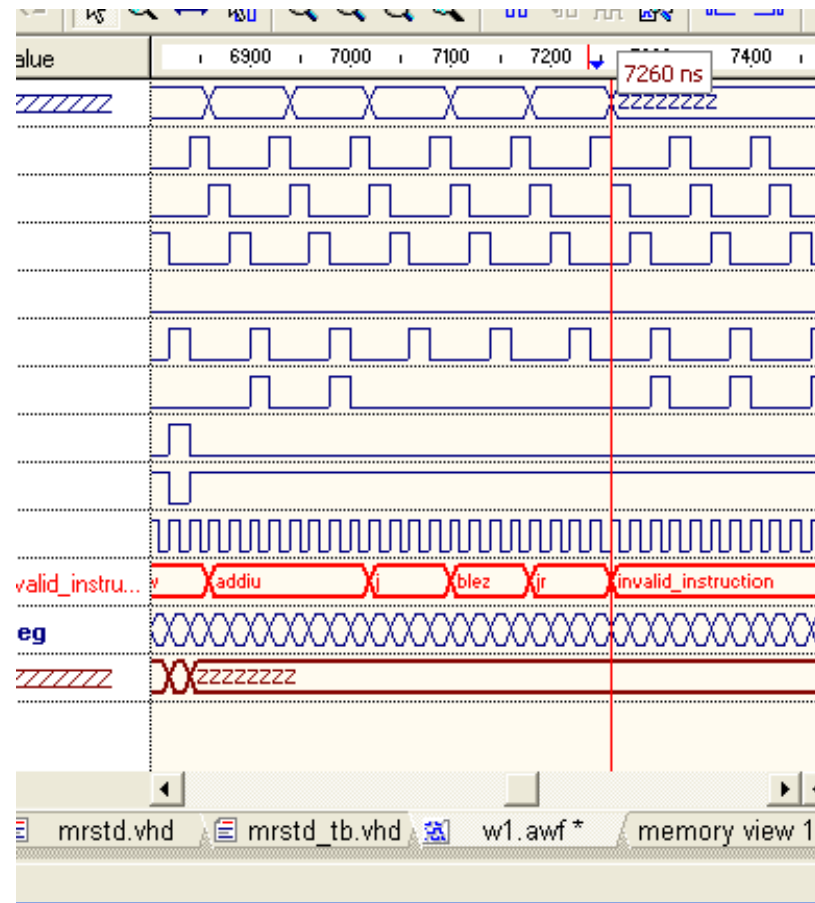
Tempo total de execução

- 7220 ns → 361 ciclos de clock

- Início: 40 ns



- Início: 7260 ns



Exercício

- Cálculo do tempo de execução
 - Considerar que se deseja utilizar o processador MIPS_S como um timer
 - Frequência do processador: 10 MHz
 - Precisão máxima do timer: 1 centésimo de segundo
 - **Pede-se:** escreva um laço que execute no tempo especificado pelo tempo do timer